

***** Inheritance

1. Inheritance is a mechanism wherein a new class is derived from an existing class.

In Java, classes may inherit or acquire the properties and methods of other classes.

A class derived from another class is called a subclass, whereas the class from which a subclass is derived is called a superclass.

2. A subclass or derived class or child class is a class which have properties of superclass or base class or parent class in addition to its own properties.

3. You can derive a class from a base class using "extend" keyword.

Syntax: `public class DerivedClass extends BaseClass {`

`}`

4. Whenever you derive a class from base class and you make any object of child class then you need to initialise the variables of base class also.

You can initialise the base class by using "super()" method.

For e.g. `super(10, 20, 30);` It will call the constructor of base class and assign the values of base class variables to a child class.

5. Derived class cannot access the private members of Base class.

// Referring a child from parent

6. Type of reference variable determines what members can be accessed not the type of object.

For e.g. `Box box5 = new BoxWeight(1,3,4,8);`

// Box is base class. BoxWeight is derived class 1,3,4 are members of base class 8 is a member of derived class

Here box5 is the reference variable of class box and the object it is pointing to, is of class BoxWeight. Hence, box5 can only access the members of itself i.e 1,3,4.

It will give error to `box5.weight(8)`.

7. // Referring a parent from child

You are given access to variables that are in the reference type i.e. BoxWeight

Hence you should have access to the weight variable only

This also means, that the ones you are trying to access should be initialised

but here, when the object itself is of type parent class, how will you call the constructor of child class

That is why error.

`BoxWeight box_weight3 = new Box(1,3,4);`

8. "super" keyword

1. It is used to call the constructor of base class.

2. It can be used as "this" keyword.

In 6th point instead of writing "this.h" you can also write "super.h". Both will give the same results

But if both base and derived class contains the variables with same name and you use

"this.variable_name" in child class it will refer to variable of child class

whereas if you use "super.variable_name" it will refer to the variable of base class.

3. When you are creating object of child class, you need to initialize variables of base class first, hence you need to use "super" keyword before initializing variables

of child class.

```
E.g. BoxWeight () {  
this.weight = weight;  
super(); // This will give an error. It should be written before "this.weight".  
}
```

The parent class' constructor needs to be called before the subclass' constructor. This will ensure that if you call any methods on the parent class in your constructor, the parent class has already been set up correctly.

In cases where a parent class has a default constructor the call to super is inserted for you automatically by the compiler.

Since every class in Java inherits from Object, objects constructor must be called somehow and it must be executed first.

The automatic insertion of super() by the compiler allows this. Enforcing super to appear first, enforces that constructor bodies are executed in the correct order which would be:

Object -> Parent -> Child -> ChildOfChild -> SoOnSoForth

***** Types of Inheritance

1. Single Inheritance:

One class extends another class.

E.g. Box --> BoxWeight

2. Multilevel Inheritance

Box --> BoxWeight --> BoxPrice

Here Box is a superclass of BoxWeight and in turn it is a superclass of BoxPrice.

3. Multiple Inheritance

class A --> class C class B --> class C

Here class C is derived from both B and A

Note: Multiple Inheritance is not allowed in Java instead we have "Interfaces" in java for the similar concept.

4. Hierarchical inheritance

class A --> class B

class A --> class C

class A --> class D

Same rules will be allowed as for single inheritance.

5. Hybrid Inheritance

Combination of single(Hierarchical) and multiple inheritance

class A --> class B class A --> class C

class B --> class D class C --> class D

Note: Since Multiple Inheritance is not allowed in java. Hence, Hybrid Inheritance is also not allowed.